

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO4 ASSESSMENT TOOL 2 – CI/CD Pipeline Design Exercise

Course Code:	CSA10 / CSA1063	Course Title:	Software Engineering
CO Assessed:	CO4	Assessment:	Assessment Tool 2 – CI/CD Pipeline Design Exercise
Weightage:	20%	Total Marks:	25
CO4 Statement:	Implement robust testing, quality assurance, and DevOps practices, emphasizing security, reliability, and ethics		
Student Name:	Tamil Elakiya S	Reg. No.:	192524443
Date:	20/08/2026	Duration:	60 minutes

Code of Conduct

I **Tamil Elakiya S (192524443)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: **Tamil Elakiya S**

Rubrics for Calculation

Criteria	Mark
Requirement & Pipeline Analysis(15%)	/5
Pipeline Architecture & Workflow(25%)	/4
Tools & Technology Selection(20%)	/4
Automated Testing & Quality Integration(15%)	/4
Deployment, Security & Rollback Strategy(15%)	/4
Pipeline Diagram & Documentation(10%)	/4
TOTAL	/25

Digital Farmer Marketplace for Agricultural Product Trading

1. Analyze the Project Requirements

1.1 Project Overview

The **Digital Farmer Marketplace** is a web-based platform that connects farmers directly with buyers. Farmers can provide agricultural products, while buyers can discover and purchase fresh agricultural products.

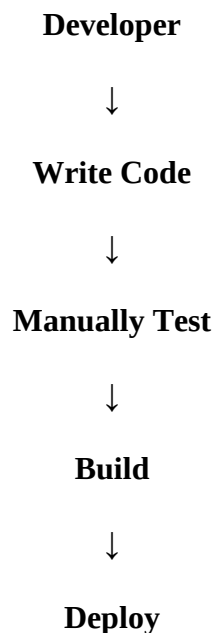
The system consists of:

- **Frontend:** React + Vite
- **Backend:** Python Flask
- **Containerization:** Docker
- **Version Control:** Git + GitHub
- **Deployment:** Docker Compose

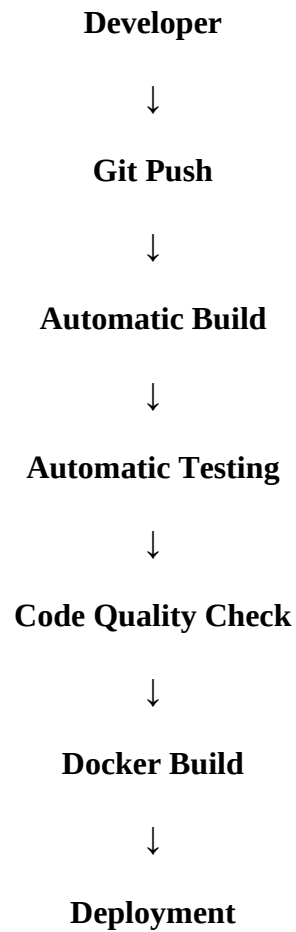
1.2 Why CI/CD is Required

A CI/CD pipeline is required to automatically verify and deploy changes whenever developers update the project.

Without CI/CD:

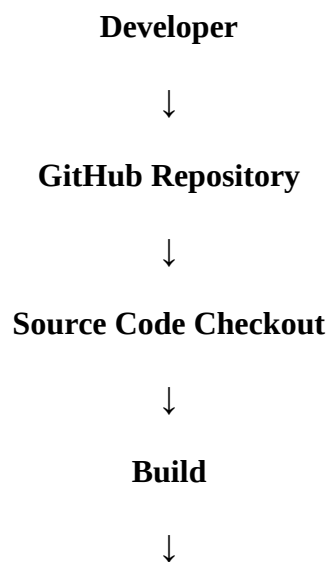


With CI/CD:



2. CI/CD Pipeline Workflow

The proposed pipeline contains the following stages:



Automated Testing



Code Quality Analysis



Docker Image Build



Security Scan



Staging Deployment



Smoke Testing



Production Deployment



Monitoring

Pipeline Stages

Stage	Purpose
Source	Retrieve latest source code
Build	Install dependencies and build application
Test	Execute automated tests
Quality	Check code quality
Security	Scan dependencies/images
Package	Create Docker images
Staging	Deploy to test environment
Validation	Perform smoke tests
Production	Deploy validated version
Monitoring	Monitor application
Rollback	Restore previous version if deployment fails

3. Tools and Technologies

Pipeline Stage	Tool
Source Control	Git
Repository	GitHub
CI/CD	GitHub Actions
Frontend Build	npm + Vite
Backend	Python + Flask
Backend Testing	pytest
Frontend Testing	Vitest
Code Quality	ESLint
Containerization	Docker
Multi-container Deployment	Docker Compose
Security	GitHub Dependabot / Trivy
Deployment	Docker-based server
Notifications	GitHub Actions

Why GitHub Actions?

GitHub Actions is suitable because your project is **already stored on GitHub**.

Therefore:

GitHub Repository



GitHub Actions



Build → Test → Scan → Deploy

No separate CI server is initially required.

4. Automated Testing Strategy

The pipeline should automatically test the application before deployment.

```
(venv) PS E:\Software engineering\Digital-Farmer-Marketplace\backend> python -m pytest ../tests/test_backend.py -v
===== test session starts =====
platform win32 -- Python 3.14.2, pytest-9.1.1, pluggy-1.6.0 -- E:\Software engineering\Digital-Farmer-Marketplace\backend\venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: E:\Software engineering\Digital-Farmer-Marketplace
collected 2 items

../tests/test_backend.py::test_home_page PASSED [ 50%]
../tests/test_backend.py::test_products_api PASSED [100%]

===== 2 passed in 24.56s =====
❖ (venv) PS E:\Software engineering\Digital-Farmer-Marketplace\backend> █
```

Figure : Successful execution of automated backend tests using Pytest

4.1 Backend Testing

Use **pytest** to test Flask APIs.

Example tests:

- ✓ Application starts
- ✓ /api/products responds
- ✓ API returns valid JSON
- ✓ Invalid API request is handled
- ✓ Product data validation works

Example:

```
def test_products_api():
    response = client.get("/api/products")

    assert response.status_code == 200
```

4.2 Frontend Testing

Use automated frontend tests to verify:

- Application renders
- Product section loads

- Buttons work
- Components render correctly
- Invalid input is handled

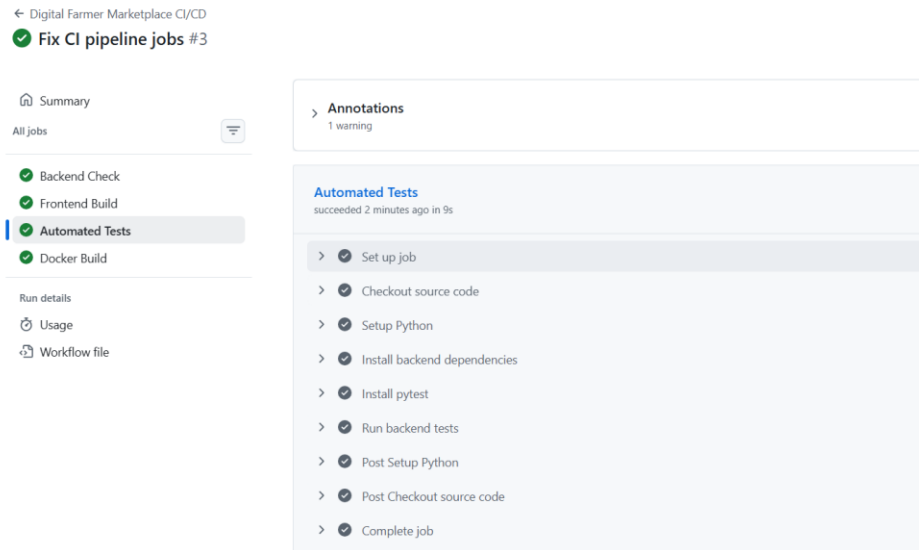


Figure: Automated backend test execution in GitHub Actions

4.3 Integration Testing

Verify communication between:

React Frontend



Flask API



Product Data

The pipeline should ensure that the frontend can communicate with the backend after deployment.

4.4 Smoke Testing

After deployment:

Open application



Homepage loads?



Backend responding?



API responding?



PASS

If the basic functionality fails, production deployment should be stopped or rolled back.

5. Deployment Strategy

Use three environments.

Development

Used by developers during implementation.

Developer



Feature Branch



Development Environment

Staging

Used for final testing before production.

Development



Automated Tests



Staging



Smoke Tests

Production

The final environment used by farmers and buyers.

Staging



Approval



Production

Environment Flow

Feature Branch



Development



Testing



Staging



Validation



Production

6. Security, Quality and Rollback

6.1 Security

The pipeline should include:

- Dependency vulnerability scanning
- Docker image scanning

- Secret management
- Environment variables for sensitive information
- No passwords/API keys committed to GitHub
- HTTPS in production
- Access control

6.2 Quality Checks

Before deployment:

Code Checkout



Linting



Unit Tests



Integration Tests



Security Scan



Docker Build

If any critical check fails:

Pipeline → FAILED

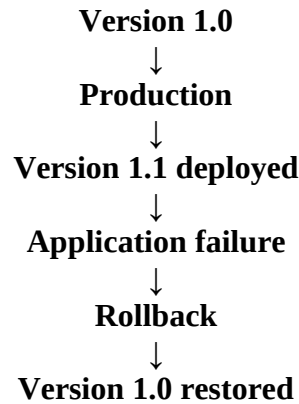


Deployment STOPPED

7. Rollback Mechanism

If the new production version fails, the previous Docker image/version can be restored.

Example:



This reduces downtime.

8. CI/CD Pipeline Diagram

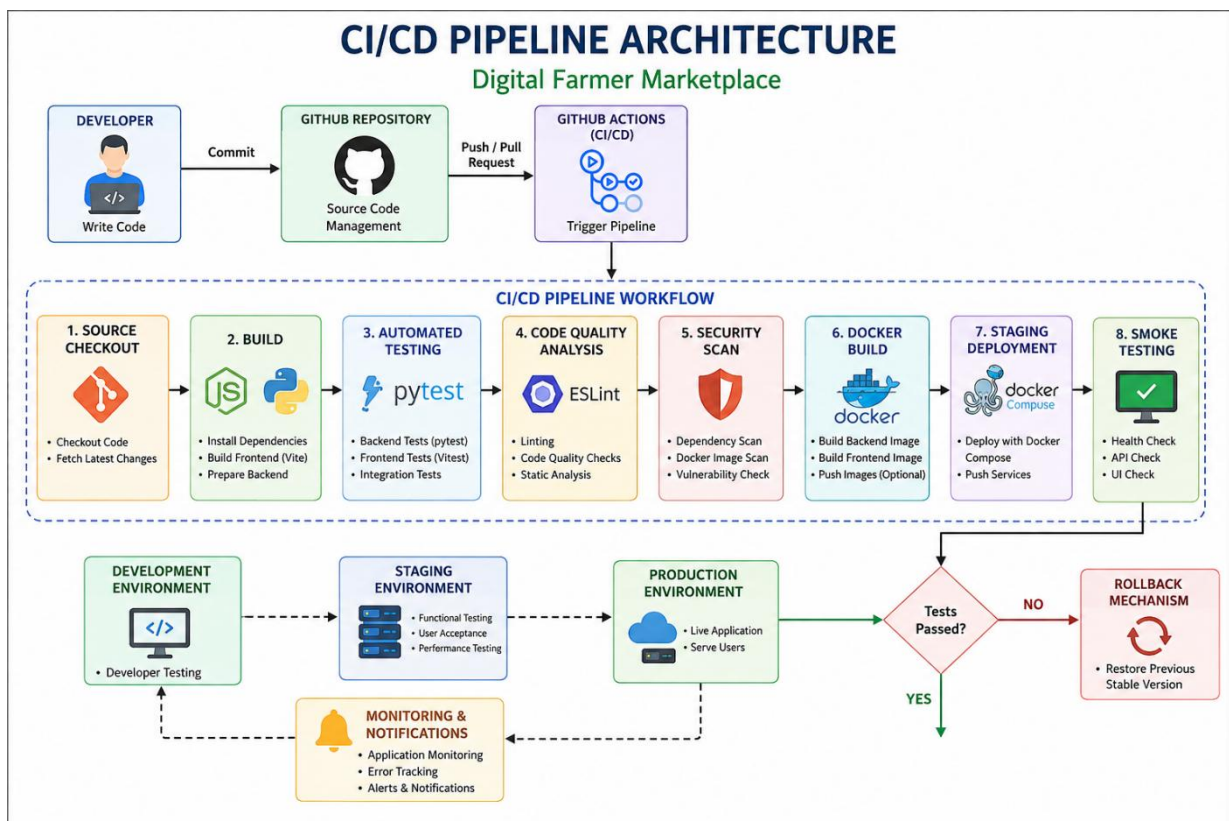


Figure : CI/CD pipeline workflow for the Digital Farmer Marketplace

The proposed CI/CD pipeline automates the process from source code commit to deployment. GitHub is used for source code management, while GitHub Actions performs build, testing, quality analysis, security scanning, Docker image creation, staging deployment, and validation. Successful builds can be promoted to production, while failed deployments can be handled using rollback mechanisms.

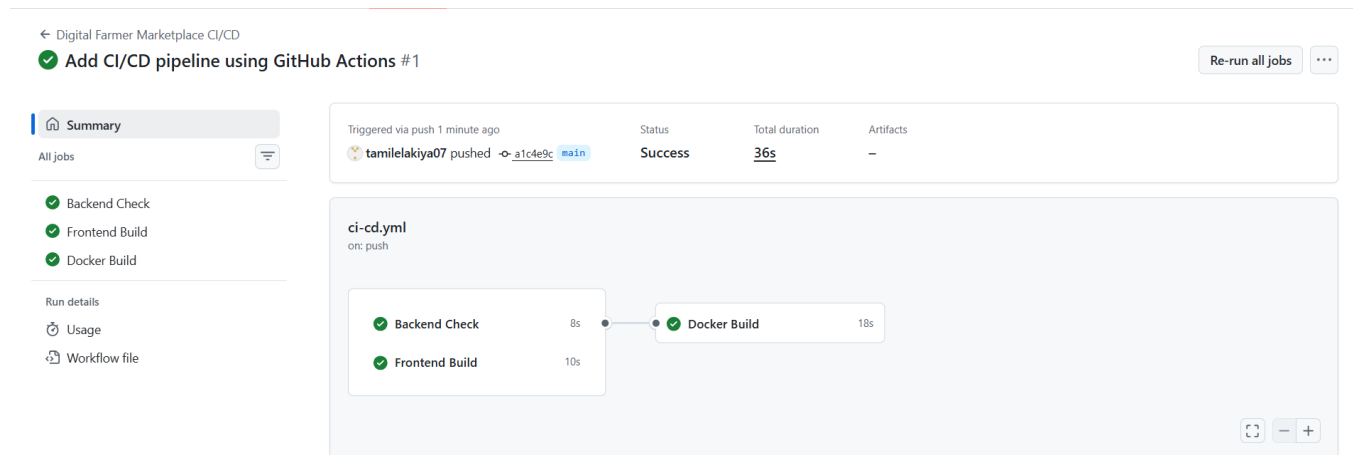


Figure : Successful execution of the Digital Farmer Marketplace CI/CD pipeline

9. Example GitHub Actions Workflow

For your actual project, create:

```
.github/  
└── workflows/  
    └── ci-cd.yml
```

A basic pipeline can be:

```
name: Digital Farmer Marketplace CI/CD
```

```
on:
```

```
  push:
```

```
    branches:
```

- main

- development

pull_request:

branches:

- main

- development

jobs:

backend-test:

runs-on: ubuntu-latest

steps:

- name: Checkout source code

uses: actions/checkout@v4

- name: Setup Python

uses: actions/setup-python@v5

with:

python-version: "3.12"

- name: Install dependencies

working-directory: backend

run: |

pip install -r requirements.txt

- name: Run backend tests

working-directory: backend

run: |

pytest

frontend-build:

runs-on: ubuntu-latest

steps:

- name: Checkout source code

uses: actions/checkout@v4

- name: Setup Node.js

uses: actions/setup-node@v4

with:

node-version: "22"

- name: Install dependencies

working-directory: frontend

run: |

npm install

- name: Build frontend

working-directory: frontend

run: |

npm run build

docker-build:

needs:

- backend-test
- frontend-build

runs-on: ubuntu-latest

steps:

- name: Checkout source code

uses: actions/checkout@v4

- name: Build Docker images

run: |

docker compose build

10. Workflow Explanation

Step 1 — Code Commit

Developer modifies the project and pushes changes to GitHub.

Step 2 — Checkout

GitHub Actions retrieves the latest source code.

Step 3 — Build

The frontend dependencies are installed and the React/Vite application is built. Backend dependencies are installed using requirements.txt.

Step 4 — Testing

Automated tests verify backend APIs and frontend functionality.

Step 5 — Quality Analysis

Linting and code-quality checks identify coding issues.

Step 6 — Security

Dependencies and Docker images are checked for vulnerabilities.

Step 7 — Docker Packaging

Frontend and backend are converted into Docker images.

Step 8 — Staging

The application is deployed using Docker Compose in the staging environment.

Step 9 — Smoke Testing

Basic application functions are verified.

Step 10 — Production

Only a successfully validated version is deployed.

Step 11 — Monitoring and Rollback

The application is monitored. If a critical failure occurs, the previous stable version is restored.

11. Justification

The proposed CI/CD approach is appropriate for the Digital Farmer Marketplace because it integrates naturally with the project's existing tools.

GitHub + GitHub Actions provides source control and automated workflows, while **React/Vite** and **Flask** can be independently built and tested. **Docker** provides consistent execution environments for both components, and **Docker Compose** simplifies multi-container deployment.

The pipeline improves:

- **Development speed** through automation
- **Code quality** through automated checks
- **Reliability** through testing
- **Security** through vulnerability scanning
- **Deployment consistency** through Docker
- **Maintainability** through structured workflows
- **Rollback capability** through versioned builds
- **Collaboration** through GitHub